

BranchBench: Aligning Database Branching with Agentic Demands

Elaine Ang, In Keun Kim, Sam Weldon, Kevin Durand, Kostis Kaffes, Eugene Wu
Columbia University



Problem

Branchable databases are becoming **infrastructure for agentic workloads** — speculative mutations and non-linear exploration. Traditional RDBMS mechanisms cannot give agents persistent isolation or concurrent branch management at this frequency.

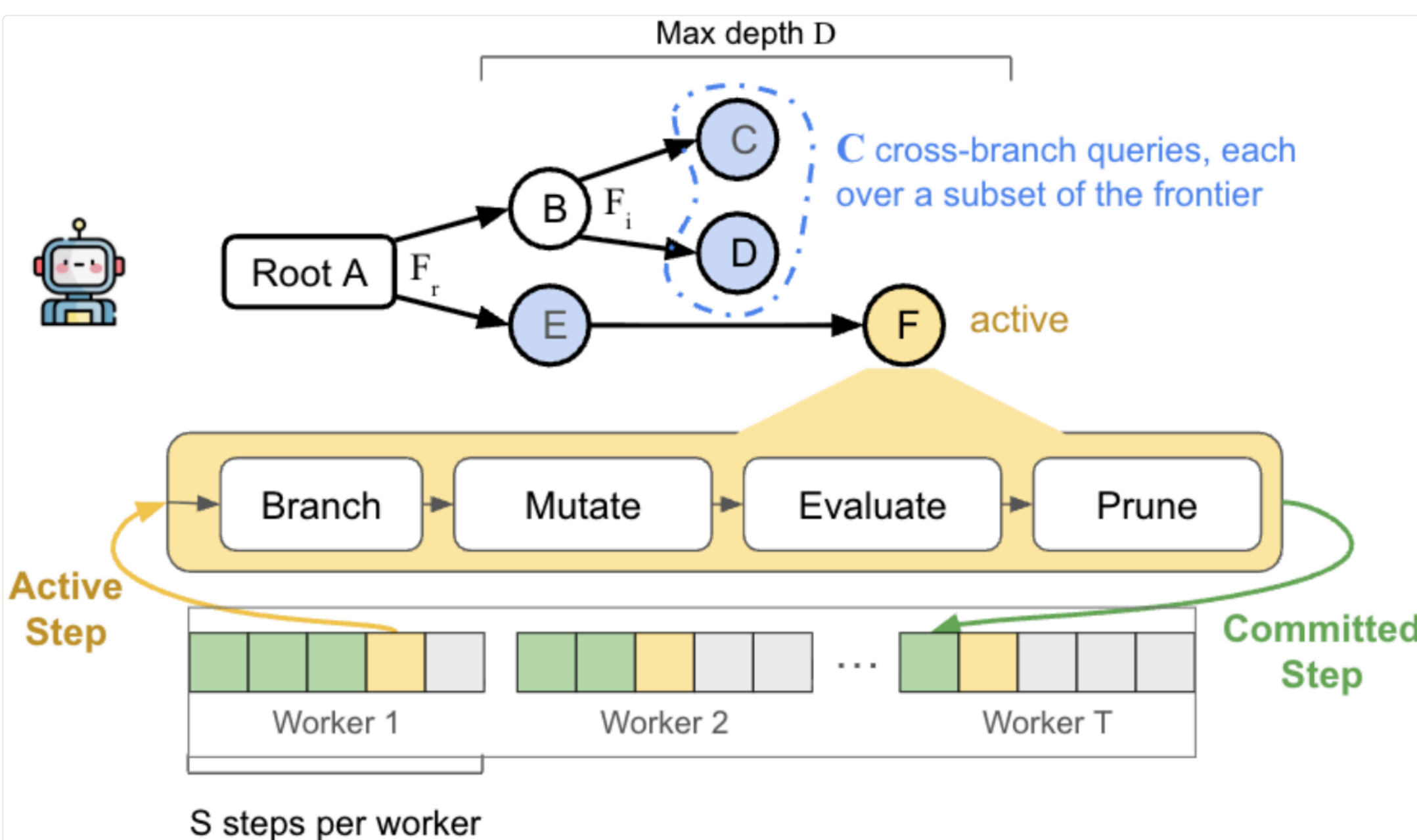
1st systematic benchmark for branchable relational DBMSes under agentic loads

Motivation

- Agentic exploration needs high-frequency **speculative state management** — checkpoint, fork, restore, evaluate, discard.
- Agents create **structurally separate branches at scale**, stressing instantiation, metadata scale, and reclamation.

An MCTS agent forks **thousands** of database branches to explore speculative actions — a regime existing branchable systems were never designed to serve.

Method



Architecture of the BranchBench macrobenchmark: the branch-mutate-evaluate-prune loop executed by parallel workers over a database state tree.

1 Four Core Operations

branch lifecycle

- Branch** — fork a writable branch below fanout and depth limits.
- Mutate** — apply data and schema ops on the active branch.
- Evaluate & Prune** — score outcomes, backpropagate reward, cut weak paths.

Neon DoltgreSQL Tiger Data Xata
PostgreSQL

2 Branch-Mutate-Evaluate Loop

per worker

Branch → Mutate → Evaluate → Prune → Commit

Each of **T** parallel workers runs **S** steps of the loop, integrating branching overhead, mutation throughput, and evaluation cost into one end-to-end score.

Microbenchmarks additionally isolate branch **creation, deletion, and persistence** costs.

3 Parameterized Workloads

5 workflows

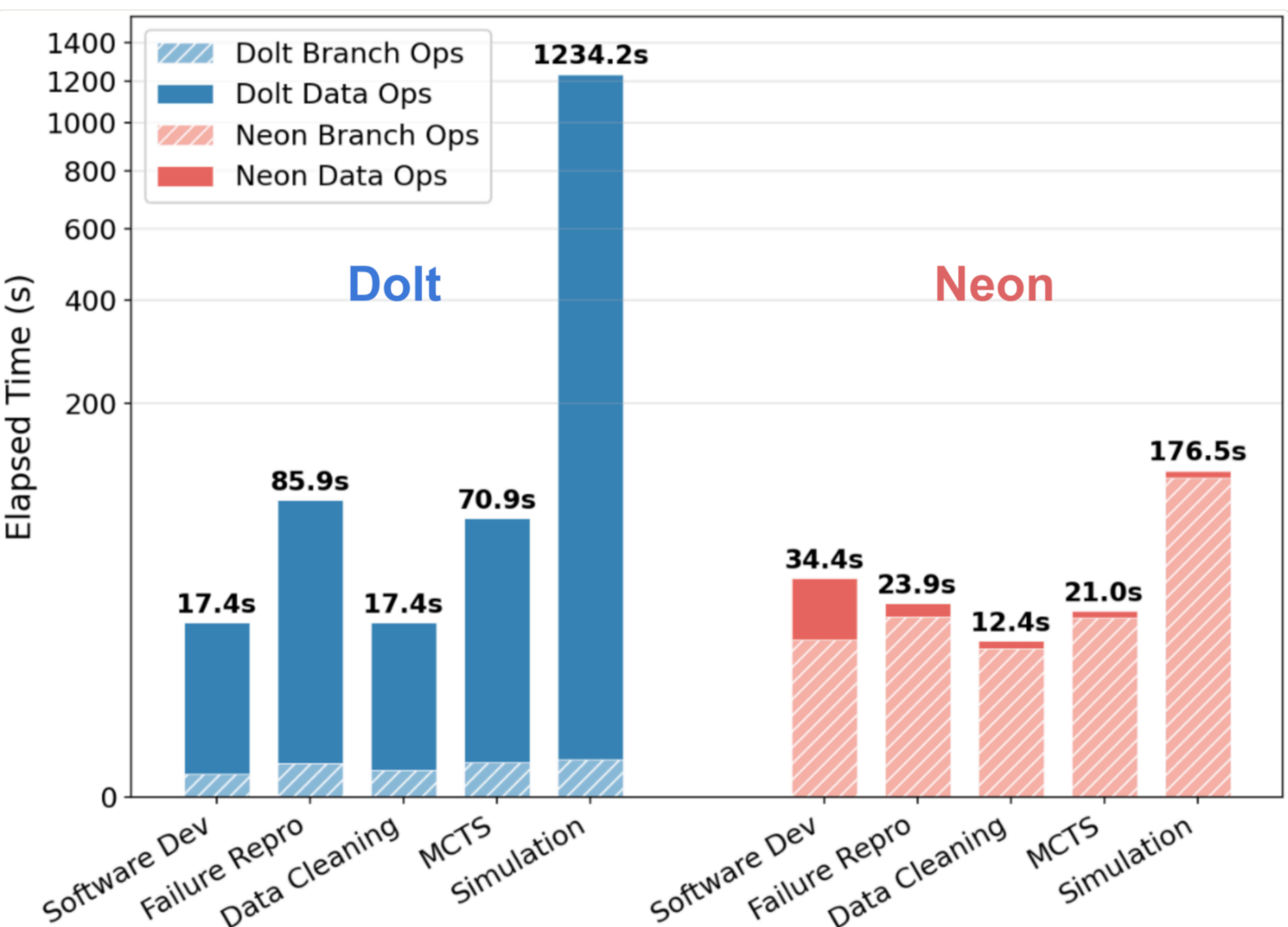
Fanout, max depth **D**, and per-step operation mix are tunable, so the loop expresses tree shapes from flat wide simulations to deep narrow MCTS under a UCT selection policy.

$$UCT(s, a) = Q(s, a) + c \sqrt{\frac{\ln N(s)}{N(s, a)}}$$

exploit reward Q vs. explore under-visited branches

4 End-to-End Workflow Cost

Dolt vs Neon



End-to-end latency per workflow (mini config): branch vs data operations across five representative agentic workflows.

Dolt's fast branch ops are dominated by slow reads; Neon stays low on data ops but pays on branch creation.

1234s
Dolt simulation

176s
Neon simulation

85.9s
Dolt failure repro

12.4s
Neon data cleaning

Read cost, not branch cost, dominates Dolt's slowest workflows (up to **1234s** on simulation) — the branch-vs-read tension is **structural**, not incidental.

Key Results

SYSTEM	BRANCH COST	READ COST
Dolt (fast-data)	low	2-4000× slower
Neon (fast-branch)	25-1500×	degrades w/ depth

A fundamental tension: no system handles all 5 workloads at scale.

SO WHAT → Fast-branching systems pay with reads up to **4000× slower** at depth; fast-data systems pay with **25-1500×** higher branch creation latency.

Headline Numbers

4000×
slower reads at depth
for fast-branching systems

1500× higher branch cost

43× Neon storage overhead

5 workloads benchmarked

Takeaway

Today's branchable databases force a choice between **fast branching** and **fast reads**, and none supports agentic workloads at scale — motivating **branch-native DBMSes** built for the high-frequency exploration autonomous agents demand.

Fast branching or fast reads — pick one. No system does both at agentic scale.